

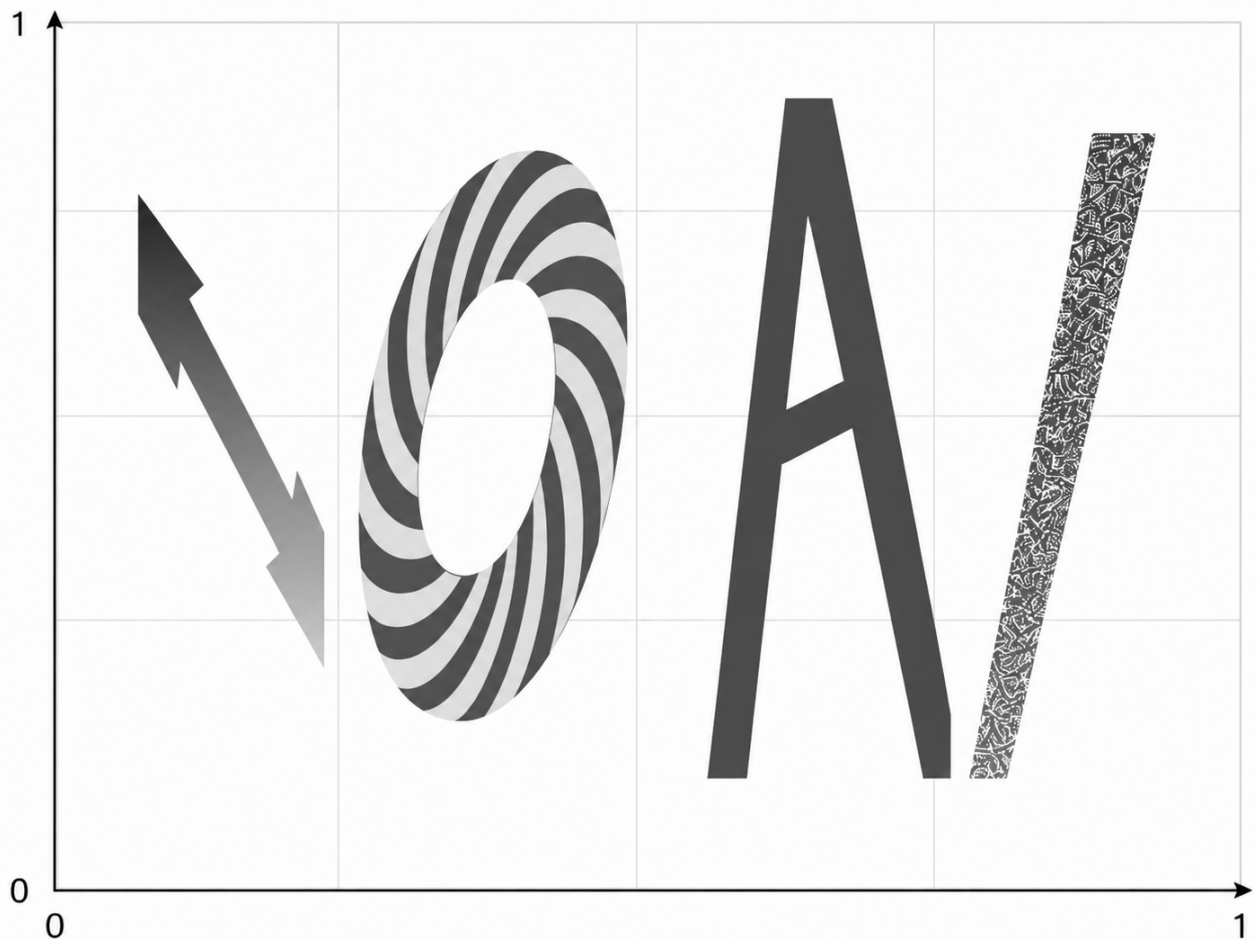
IOAI դաշտ

- **Time limit:** 5 minutes
- **Storage:** 5 GB
- **Solution size:** `solution.ipynb`, `custom_model.py` ≤ 1 MB together
- **Pretrained models:** չկան — train from scratch, գնահատման ընթացքում ինտերնետ չկա
- **Baseline Score:** 31.2187

Առաջադրանք

Աստանայի քաղաքապետը ցանկանում է զարդարել քաղաքը ոճավորված IOAI լոգոներով: Որպես վիճակագիր, նա ամեն ինչ՝ ներառյալ լոգոն, դիտարկում է որպես տարածական ֆունկցիա $F(x, y, \overline{W})$, որտեղ $x, y \in [0, 1]$ -ը ներկայացնում են կոորդինատները 2D հարթության վրա, իսկ $\overline{W}$ -ը թաքնված պարամետրերի բազմությունն է, որոնք սահմանում են ոճական հատկանիշներ, ինչպիսիք են տառերի գույները և անկյունները:

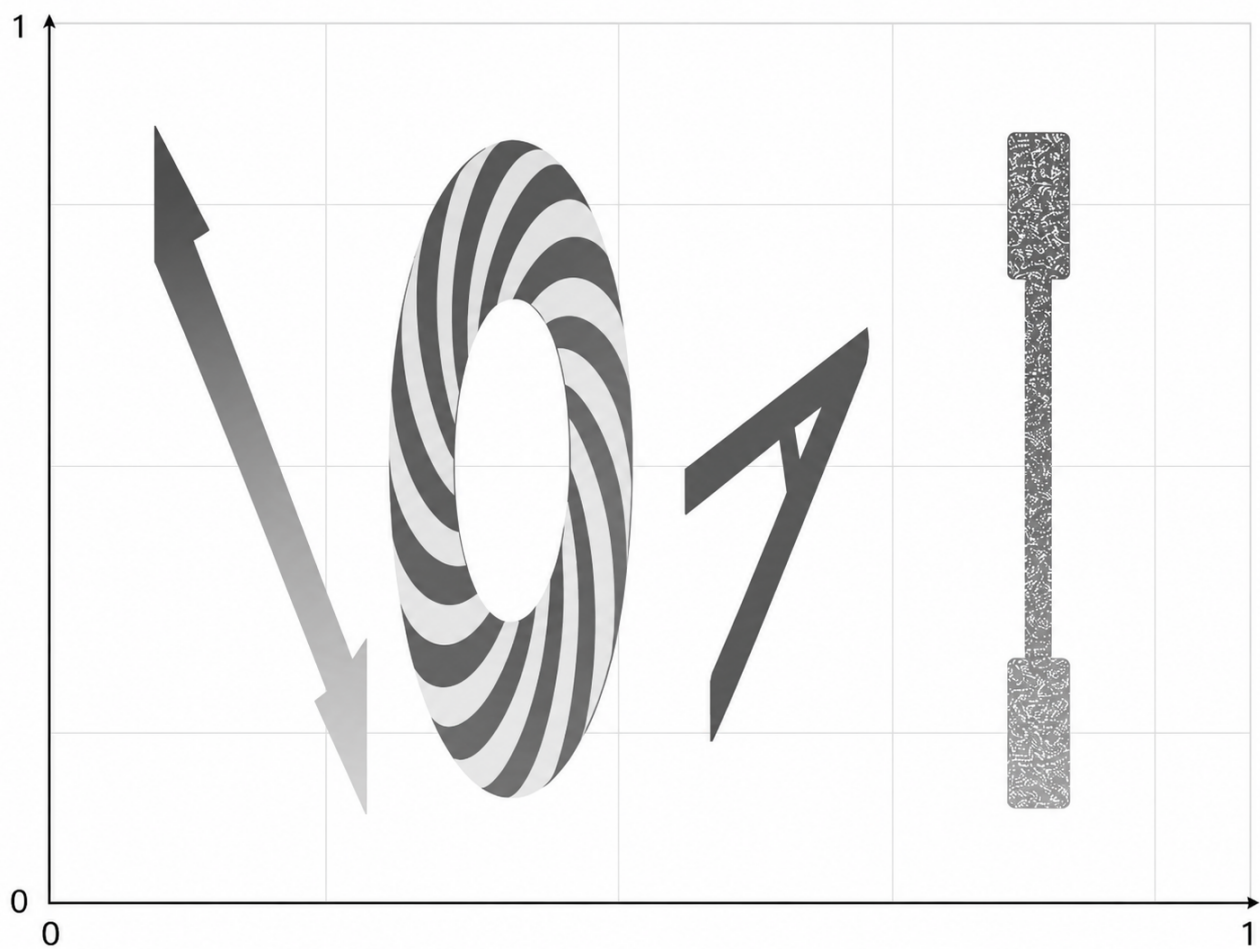
Քանի որ F -ը չափազանց բարդ է բացահայտ մաթեմատիկական հավասարման տեսքով արտահայտելու համար, ձեր առաջադրանքն է train անել ներդրոնային ցանց՝ այն մոտարկելու համար: Ցանցը ցանկացած (x, y) կոորդինատային զույգի համար կվերադարձնի **IOAI field**-ի արժեք՝ հարթության վրա գեներացնելով լոգոյի ամբողջական heatmap վիզուալիզացիան: Ահա F -ի heatmap վիզուալիզացիայի օրինակ որոշակի թաքնված $\overline{W}$ պարամետրերով:

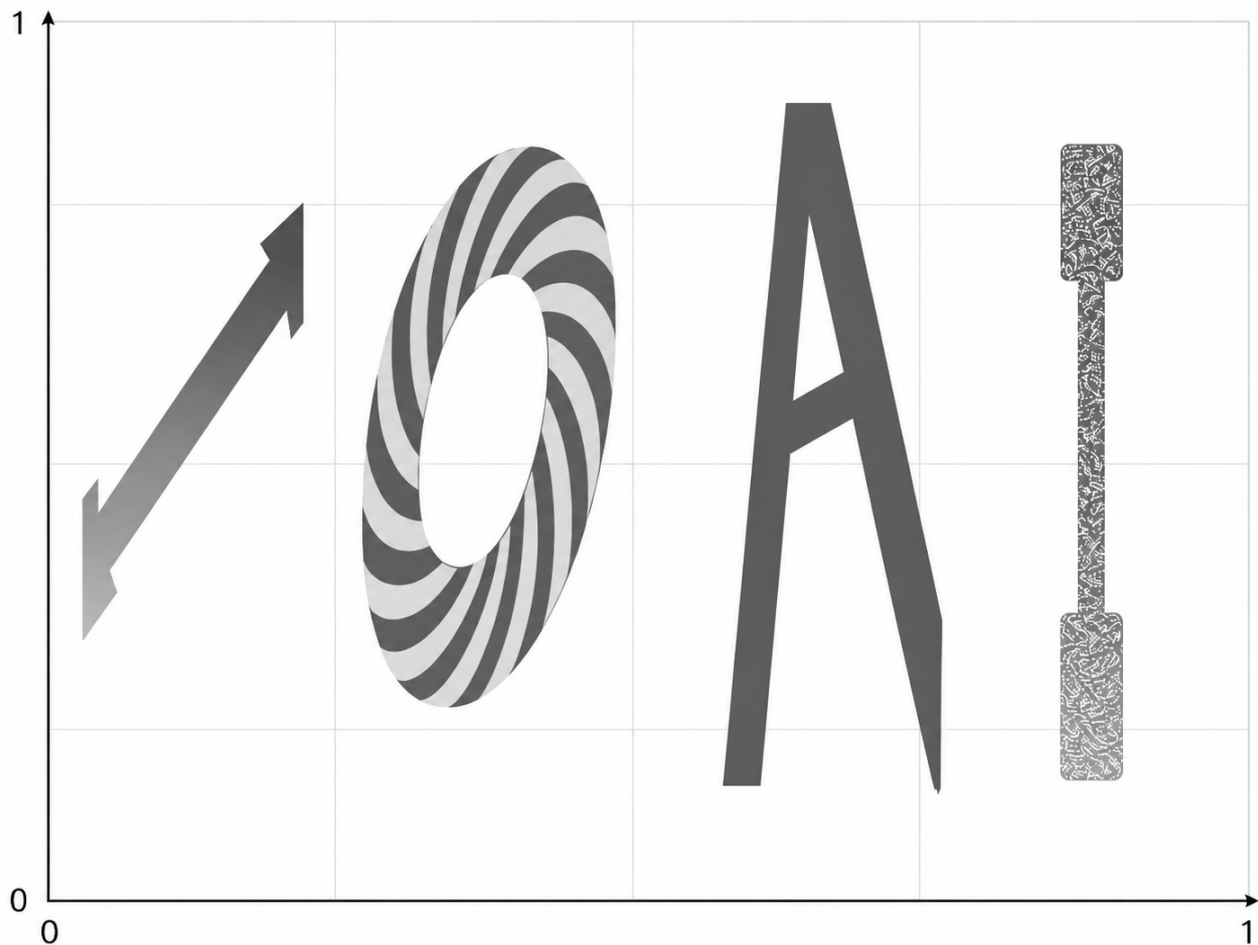


Ինչի՞ց է բաղկացած IOAI field-ը: Չորս տառից և ֆոնից (background):

- Առաջին՝ Γ տառի ներսում արժեքները շատ մեծ են ($1e+10$ և ավելի) գծային գրադիենտով
- $\bigcirc$ տառի ներսում արժեքները ցուցադրում են սպիրալաձև pattern
- A տառի ներսում արժեքը միշտ -1 է
- Վերջին՝ Γ տառի ներսում արժեքները պետք է լինեն պատահական արժեքներ $[-2026, 2026]$ միջակայքից, նույնիսկ եթե գնահատվում են նույն կետում երկու անգամ
- Տառերից դուրս արժեքը միշտ զրո է

Ֆունկցիան ունի թաքնված $\overline{W}$ պարամետրեր, որոնք ազդում են տառերի scale-ի (միջակայքի) և incline-ի վրա, ինչպես նաև առաջին Γ տառի ներսում արժեքների միջակայքի վրա: Այնուամենայնիվ, տառերը չեն հատվի: Ահա մի քանի օրինակներ, թե ինչ տեսք ունի IOAI field-ը տարբեր $\overline{W}$ -ների դեպքում.





Ինչ է տրված ձեզ.

Այս խնդիրը ՉԻ պարունակում dataset-ներ: Փոխարենը, ձեզ տրված է generator ֆունկցիան, որը կոնֆիգուրացված է `data/train_config/field_config.json` հասցեում գտնվող JSON config ֆայլով:

Թեստային config-ը թաքցված է, բայց այն նմանատիպ բնույթ ունի: Ձեր առաջադրանքն է fit լինել տրված generator-ի վրա՝ օգտագործելով այնքան տվյալ, որքան ցանկանում եք: Ձեր "train" և "test" բաշխումները գեներացվում են նույն generator-ից. դուք պարզապես չգիտեք, թե որ (x_i, y_i) կետերում եք գնահատվելու:

Ձեր submission-ը պետք է բաղկացած լինի հետևյալից.

- training model դաս՝ պահպանված որպես `custom_model.py`: Այս մոդելը պետք է ժառանգի `torch.nn.Module` դասից և օգտագործի միայն `torch` import-ներ: Այն պետք է պարունակի `CustomModel` դասը, որն օգտագործվում է `solution.ipynb` notebook-ում:
- `solution.ipynb` notebook, որը կստեղծի `model.pt` weights-ը

Գնահատում

Յուրաքանչյուր տիրույթի համար նվազագույն score-ը 0 է, իսկ առավելագույն score-ը՝ 1: Վերջնական score-ը հաշվարկվում է բոլոր հինգ տիրույթների (չորսը՝ յուրաքանչյուր տառի համար, և ֆոնը) միջինացմամբ և բազմապատկվում 100-ով: Առկա է պարամետրերի տուգանք (parameter penalty):

Եթե ձեր մոդելն ունի ավելի քան 20260 պարամետր, score-ը կհստկ չափ նվազեցվում է:

Պարամետրերի քանակը չափվում է `sum(p.numel() for p in model.parameters())`-ով: Մենք ակնկալում ենք, որ ձեր մոդելը կաշխատի նաև ստոխաստիկ ռեժիմում, ընդ որում PyTorch `nn.Dropout`-ը կլինի մոդելի մաս:

Ստանդարտ տիրույթների համար

Յուրաքանչյուր R տիրույթի համար (առաջին I տառը, `O`, `A`, `Background`), մենք գնահատում ենք մոդելը $N_R = 512$ test կետերի (x_i, y_i) վրա՝ v_i իրական արժեքներով և $\hat{v}_i$ կանխատեսումներով: Որպես հիմնական մետրիկա՝ մենք օգտագործում ենք նորմալացված Mean Absolute Error (MAE): MAE-ն սահմանվում է որպես.

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

Իսկ նորմալացումն իրականացվում է որպես.

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

որտեղ $s_R > 0$ -ը scale-ի հաստատուն է:

Վերջին I տառի տիրույթի համար

Այս տիրույթում գնահատման ընթացքում dropout-ն ակտիվացված է: Յուրաքանչյուր j test կետի համար.

1. Մենք աշխատեցնում ենք մոդելը $K = 10$ անգամ՝ ստանալու համար $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$:
2. Եթե որևէ ելք դուրս է $[-2026, 2026]$ միջակայքից, ապա $\text{pointScore}(j) = 0$:
3. Հակառակ դեպքում, հաշվարկեք K ելքերի σ_j ստանդարտ շեղումը և փոխակերպեք այն score-ի.

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

որտեղ $s_E > 0$ -ը scale-ի ֆիքսված հաստատուն է:

Տիրույթի score-ը այդ տիրույթի բոլոր կետերի ըստ միջինի արժեքն է.

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

որտեղ $N_E = K * N_R$:

Պարզ ասած, որքան ավելի շատ բազմազանություն ունենաք, այնքան ավելի մեծ կլինի ձեր score-ը այս տիրույթի համար: **Դուք չեք կարող օգտագործել random մաքուր տեսքով, ներառյալ PyTorch rand* և _uniform ֆունկցիաները. պատահականությունը պետք է ստացվի ակտիվացված dropout-ով inference-ից:**

Ինչպես ուղարկել լուծումը

1. Բացեք `solution.ipynb`-ը և աշխատեցրեք բոլոր cell-երը:
2. Բարելավեք `CustomModel` մոդելը `custom_model.py`-ում:
3. Համոզվեք, որ ձեր վերջին cell-ը պահպանում է ձեր մոդելը `model.pt` ֆայլում:
4. JupyterLab-ի `Git` tab-ում `stage` արեք, `comment` ավելացրեք և `commit` արեք `solution.ipynb` ու `custom_model.py` ֆայլերը, ապա `push` արեք դրանք:
5. Վերադարձեք `Contest` էջ և սեղմեք **Submit**: `Submit`-ի `comment`-ը պետք է նույնը լինի, ինչ նախորդ քայլի `comment`-ը: